Compresr SDK · Python

LangChain Integration

Auto-compress tool outputs in LangChain agents to cut tokens, latency, and cost — drop-in middleware, no app rewrites.

1 Why teams use it

Agents waste tokens on bulky tool outputs — search results, CRM dumps, scraped pages, SQL rows — that the LLM only needs a small slice of. Compresr rewrites each tool output *against the user's query* before it re-enters the conversation, so the model sees only what matters.

- **Lower bills** — typically 40–80% fewer tokens per tool turn.
- **Faster turns** — smaller prompts mean faster LLM responses.
- **Longer agent loops** — stay under context limits on multi-step runs.
- **Zero refactor** — one line in `create_agent`; your tools stay untouched.

Compresr fails open by default: if the compression API is unreachable, your agent keeps running on the original tool output — never a hard stop.

2 Install

```
pip install 'compresr[langchain]'
export COMPRESR_API_KEY=cmp_...
```

Requires LangChain 1.0+ and Python 3.10+.

3 Drop-in middleware

Pass `CompresrToolMiddleware` to `create_agent`. Every tool output is compressed against the user's query before the next LLM turn — no tool code changes.

```
from langchain.agents import create_agent
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI
from compresr.integrations.langchain import CompresrToolMiddleware

@tool
def web_search(query: str) -> str:
    # your real search call here
    return "<search results...>"

agent = create_agent(
    model=ChatOpenAI(model="gpt-4o-mini"),
    tools=[web_search],
    middleware=[CompresrToolMiddleware(
        target_compression_ratio=0.6,   # remove ~60% of tokens
        min_tokens=200,                 # leave short outputs alone
        query_arg="query",              # use this tool arg as the query
    )],
)

result = agent.invoke({"messages": [
    {"role": "user", "content": "Find recent pricing complaints."},
]})
```

That's it. Tool outputs flow through Compresr; everything else — message shape, tool-call IDs, error handling — is preserved.

4 Wrap a single tool

When you only want to compress one noisy tool (and skip the middleware entirely), wrap it directly:

```
from langchain_core.tools import tool
from compresr.integrations.langchain import wrap_tool_with_compression

@tool
def search_crm(query: str) -> str:
    """Search the CRM for customer notes."""
    return "<CRM rows...>"

search_crm = wrap_tool_with_compression(
    search_crm,
    target_compression_ratio=0.7,
    query_arg="query",
)
```

Works with plain LCEL, custom LangGraph graphs, or anywhere a `StructuredTool` is used. Both sync and async paths are covered.

5 Key parameters

Parameter	Default	Description
`api_key`	`None`	Compresr API key. Falls back to `COMPRESR_API_KEY`.

Parameter	Default	Description
`target_compression_ratio`	`0.5`	Compression strength. $\in [0, 1]$ = fraction of tokens to remove (`0.6` = remove 60%); > 1 = Nx factor (`4` = 4x smaller).
`min_tokens`	`200`	Skip outputs below this token count — don't pay to compress what's already small.
`compression_model`	`latte_v1`	Compression model. `latte_v1` is the default; `latte_v2` is the faster variant.
`query`	`None`	Static query string — useful when every call shares the same intent.
`query_arg`	`None`	Name of the tool argument whose value should be used as the query (e.g. `"query"` , `"question"`).
`query_extractor`	`None`	Callable for custom query logic — return `None` to fall back to the default.

Query resolution order: `query` → `query_extractor` → `query_arg` → smart default (most recent user message). You only need to set one.

Also available: `CompresrSummarizationMiddleware` (KV-cache-friendly history summarization) and `CompresrPromptMiddleware` (last-mile prompt budget enforcement). A `BaseDocumentCompressor` for retriever-side compression ships as `CompresrExtractor` .